Kakeibo — Mindful Money Android App

Product Requirements Document & Technical Architecture

Field	Value
Document version	1.2 (LKR currency + symbol format setting)
Date	3 November 2026
Platform	Android (APK), offline-first
Status	Ready for review

Table of Contents

1. Research: What Is Kakeibo?
2. Product Vision & Goals (incl. Assumptions & Glossary)
3. Target Users & Personas
4. Competitive Landscape
5. Scope (MVP vs Later)
6. Functional Requirements
7. User Flows
8. Screen Inventory
9. Non-Functional Requirements
10. Technical Architecture
11. Data Model
12. Core Business Logic
13. Security & Privacy
14. Testing Strategy
15. Build, Release & APK Distribution
16. Roadmap & Milestones
17. Success Metrics
18. Risks & Mitigations
19. Open Questions
20. Sources

1. Research: What Is Kakeibo?

1.1 Origin

Kakeibo (家計簿) literally means *"household account book."* It was created in **1904 by Hani Motoko**, widely described as Japan's first female journalist, as a simple ledger to help households manage money. It remains popular in Japan more than a century later, and was popularised internationally by Fumiko Chiba's book *Kakeibo: The Japanese Art of Budgeting & Saving Money*.

1.2 The core philosophy

Unlike rule-based budgets (50/30/20, zero-based, envelope), Kakeibo is a **mindfulness practice** more than a strict spending cap. Its key ideas:

- **Awareness over automation** — the act of writing each expense by hand forces you to notice it. Traditional Kakeibo is handwritten because the physical act reinforces awareness.
- **Save first, spend second** — decide your savings goal *before* the month begins.
- **Reflect regularly** — a weekly glance and an honest end-of-month review are the heart of the method.
- **Improve one thing at a time** — each month ends with one concrete change for the next.

1.3 The four questions

Every month is framed by four questions:

#	Question	When
1	How much money do I have available?	Start of month
2	How much would I like to save?	Start of month
3	How much am I actually spending?	During / end of month
4	How can I improve?	End of month

1.4 The four spending categories

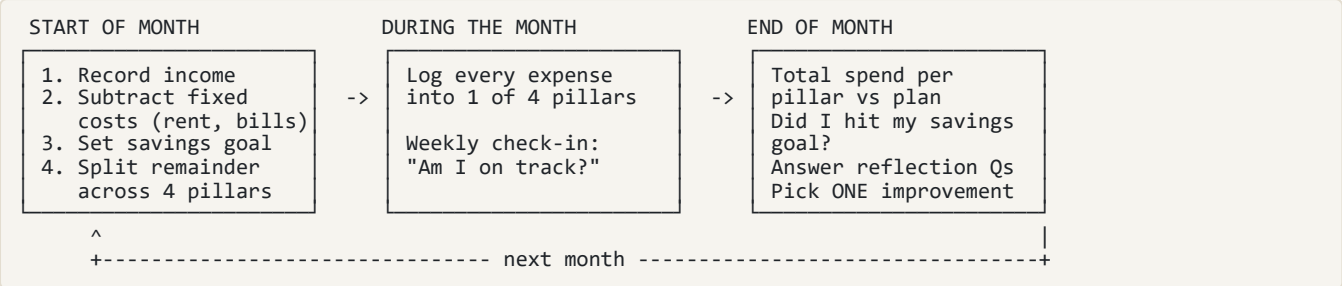
Every variable expense goes into exactly one of four "pillars":

Category	Japanese idea	Examples
Survival / Needs	Essentials to live and work	Groceries, transport, medicine, utilities top-ups
Optional / Wants	Nice to have, not required	Eating out, shopping, takeaway coffee
Culture	Things that grow your mind	Books, courses, museums, music, streaming
Unexpected / Extra	Unplanned or one-off	Repairs, gifts, medical emergencies, fines

> Sources word these differently (e.g. "Needs, Wants, Culture, Unexpected" vs "Essential, Optional, Entertainment, Unforeseen"). The app will use **Needs · Wants · Culture · Unexpected** and allow users to

> rename labels.

1.5 The monthly cycle



Formula at the heart of Kakeibo:

```
Spendable = Income − Fixed Expenses − Savings Goal
Actual Savings = Income − Fixed Expenses − Σ(Variable Expenses)
```

1.6 Reflection prompts (end of month)

- Did I achieve my savings goal? Can I repeat it? Could I aim higher?
- Which pillar surprised me the most?
- Which purchases do I regret? Which made me genuinely happy?
- What is the ONE thing I will change next month?

1.7 Strengths and weaknesses (from research)

Strengths	Weaknesses
Builds lasting spending awareness	Manual logging can feel tedious, especially with many small transactions
Simple — only 4 categories	Easy to abandon after a few weeks
No bank linking → private	No automatic bank import
Emotional reflection, not just numbers	Paper books cannot chart trends or remind you

1.8 Product insight

> **The opportunity:** keep the *intentional friction* that makes Kakeibo work (manual entry + reflection), but remove the *useless friction* (math, forgetting, losing the notebook, no history). A great Kakeibo app should make logging take **under 5 seconds** while still requiring a conscious choice of pillar — and make the monthly reflection a delightful ritual instead of a chore.

---

2. Product Vision & Goals (incl. Assumptions & Glossary)

Vision: *A calm, private, offline pocket ledger that helps me spend with intention and save more every month — the Kakeibo way.*

2.1 Goals

ID	Goal	Measure
G1	Help the user save money every month	% of months savings goal met
G2	Build a daily logging habit	Days with ≥1 entry / days in month
G3	Make the reflection ritual stick	% of months with completed review
G4	Keep data 100% private and offline	No network permission required for core features

2.2 Non-goals (v1)

- Bank account / SMS auto-import (conflicts with the mindful-entry philosophy; may be optional later)
- Investment tracking, stock portfolios, credit scores
- Multi-user cloud sync / shared household accounts (later)
- iOS build (architecture keeps it possible)

2.3 Assumptions & Constraints

- Single user, single device, single currency per installation (v1). **Default currency: Sri Lankan Rupee (LKR)**, shown as `Rs` by default; the user lives in Sri Lanka.
- The user is paid roughly monthly; a "period" is one month starting on a chosen day (1–28).
- No server, no account, no internet permission — therefore **no remote recovery** of PIN or data; backups are the user's responsibility, so the app must make them easy and automatic.
- Distributed as a sideloaded, self-signed APK first; Play Store later.
- Built by one developer; MVP features must be achievable in ~6 weeks.
- **Savings carry-over decision (resolves former Open Question 2):** savings are tracked **per period** (goal vs actual) **and** as a running lifetime total. Unspent spendable money at period end is counted as *extra savings* of that period; overspending reduces that period's actual savings (which may go negative). Pillar budgets do **not** roll over.

2.4 Glossary

Term	Meaning
Period	One budgeting month from `start_date` to `end_date` (inclusive), based on month start day
Pillar	One of the 4 Kakeibo spending categories: Needs, Wants, Culture, Unexpected
Income	Money received in the period (planned salary + extra income logged mid-month)
Fixed cost	Predictable obligation paid every period (rent, EMI, subscriptions) — not logged as pillar expenses
Variable expense	Any day-to-day spend logged into a pillar
Savings goal	Amount (or % of income) the user commits to save *before* spending
Spendable	Income − Fixed − Savings goal: what can be spent across the 4 pillars
Remaining	Spendable − variable expenses so far
Daily allowance	Remaining ÷ days left in the period (never negative)

Term	Meaning
Actual savings	Income − Fixed − variable expenses (net of refunds)
Reflection	End-of-period review answering the Kakeibo questions + one improvement
Improvement	The single concrete change chosen in a reflection, tracked during the next period
Partial period	A first period that starts mid-month (from install day) with pro-rated amounts

3. Target Users & Personas

Persona A — "The Saver-to-be" (primary: the app owner) Young professional, salaried, feels money "disappears" each month. Wants to see where it goes and save a fixed amount. Uses Android daily. Prefers privacy, does not want to link a bank account.

Persona B — "The Overwhelmed Budgeter" Tried complex apps (YNAB, Mint-style) and gave up due to too many categories. Wants simplicity — 4 buckets only.

Persona C — "The Reflective Journaler" Likes bullet journals and habit trackers. Values the monthly reflection and notes more than charts.

4. Competitive Landscape

App / Approach	Strength	Gap we fill
Paper Kakeibo notebook	Mindful, tactile	No math, reminders, history, charts
Generic expense trackers	Many features	Dozens of categories, no reflection ritual, often ads/cloud
Bank-linked apps	Automatic	Passive — removes the awareness Kakeibo depends on; privacy concerns
Existing Kakeibo apps (iOS-focused, small)	Follow the method	Few polished, free, offline Android options with a guided ritual

Differentiators: guided monthly ritual · 4-pillar simplicity · 5-second quick-add · fully offline & private · "regret/joy" emotion tag on each expense · one-improvement goal tracking.

5. Scope

5.1 MVP (v1.0)

1. Onboarding (currency, month start day, pillar names) **with an interactive app guide** (Kakeibo lessons, guided first entry, coach marks, Help Center)
2. Monthly setup wizard (income, fixed costs, savings goal, pillar budgets)
3. Quick-add expense (amount, pillar, note, date, optional mood)

4. Home dashboard (remaining spendable, savings progress, pillar bars)
5. Transactions list with search / filter / edit / delete
6. Weekly check-in summary
7. End-of-month reflection wizard (4 questions + improvement)
8. History & simple charts (month-over-month)
9. Reminders (daily log nudge, weekly check-in, month-end review)
10. Local backup / restore (JSON file), automatic weekly backup, CSV export, delete all data
11. App lock (PIN / biometric) with recovery code
12. Light / dark theme
13. "My stats" screen (habit & savings metrics computed locally)

5.3 MVP traceability

MVP item	Requirements
1. Onboarding + guide	FR-01 – FR-05, FR-90 – FR-108, FR-109
2. Monthly setup	FR-10 – FR-18
3. Quick-add	FR-19 – FR-28
4. Dashboard	FR-30 – FR-35
5. Transactions	FR-24, FR-84
6. Weekly check-in	FR-40 – FR-42
7. Reflection	FR-50 – FR-58
8. History & charts	FR-60 – FR-63
9. Reminders	FR-80 – FR-83, FR-85, FR-86
10. Data management	FR-70 – FR-75
11. App lock	FR-06 – FR-09
12. Theme	FR-05
13. My stats	FR-36

Every MVP item maps to at least one **P0** requirement; P1 items inside an MVP feature may slip to v1.1 without blocking release.

5.2 Later (v1.x – v2)

- Home-screen widget for quick-add
- Recurring fixed expenses templates & income splits
- Savings "jars" for long-term goals (e.g. trip, emergency fund)
- Multiple income sources & multiple currencies
- Optional encrypted cloud backup (Google Drive)
- Optional SMS/notification parsing → *suggested* entries that still need confirmation
- Shared household ledger
- Streaks & gentle achievements
- iOS build

6. Functional Requirements

Priority: **P0** = MVP must-have, **P1** = MVP nice-to-have, **P2** = later.

6.1 Onboarding & Settings

ID	Requirement	Priority
FR-01	Welcome guide explaining Kakeibo and the app (see §6.10, FR-90 – FR-107)	P0
FR-02	Currency is **LKR (Sri Lankan Rupee)** by default, displayed as `Rs 12,500.00`; confirmed (not chosen from a long list) during onboarding	P0
FR-03	Choose month start day (1–28, e.g. salary day)	P0
FR-04	Rename pillar labels, pick colour/icon	P0
FR-05	Theme: system / light / dark	P0
FR-06	App lock via PIN and/or biometric	P0
FR-07	At PIN setup, show a one-time **recovery code** (12 chars) the user must confirm by re-typing; a forgotten PIN can be bypassed by biometric or recovery code. Without either, the only option is **Reset app** (wipe) with a clear prompt to restore from backup	P0
FR-08	After 5 wrong PIN attempts, enforce increasing lockout (30 s, 1 min, 5 min, …)	P0
FR-109	**Currency display format** (Settings → Money → Currency format): change only how the symbol is shown — `Rs` · `Rs.` · `LKR` · ₹ · ৳ · Custom (1–4 chars); position before/after the number; space on/off; live preview (e.g. `Rs 12,500.00`). Changing it **never converts or alters stored amounts** and applies instantly on every screen, notification and help text	P0
FR-09	Auto-lock after configurable inactivity (immediate / 1 / 5 min) and re-lock on resume after timeout; optional **hide in app switcher** (`FLAG_SECURE`) toggle	P0

6.2 Monthly Setup (Questions 1 & 2)

ID	Requirement	Priority
FR-10	At the start of each period, prompt the user to plan the month	P0
FR-11	Enter one or more income entries for the month	P0
FR-12	Enter fixed expenses (rent, EMI, subscriptions); copy from last month in one tap	P0
FR-13	Enter savings goal (amount or % of income)	P0
FR-14	App shows `Spendable = Income − Fixed − Savings` live	P0
FR-15	Optionally split spendable across 4 pillars (suggested split pre-filled from history)	P1
FR-16	Plan can be edited anytime during the month; changes are logged in `plan_changes`	P1
FR-17	**Mid-month first install:** user chooses (a) *partial period from today* — income/fixed/savings pre-filled pro-rated by days remaining, editable; or (b) *full period* — enter past spending as one "Before I started" lump sum per pillar or as backdated entries	P0

ID	Requirement	Priority
FR-18	**Save first:** "Have you set aside your savings?" checkbox on the plan screen and dashboard until ticked (Kakeibo pays-yourself-first principle)	P0
FR-19	Manageable payment-method list (Cash, UPI, Card, Bank by default; add/rename/hide)	P1

6.3 Expense Logging (Question 3)

ID	Requirement	Priority
FR-20	Quick-add from a floating button; numeric keypad opens immediately	P0
FR-21	Must choose exactly one pillar (big 4-button selector)	P0
FR-22	Optional note, date (defaults today), payment method	P0
FR-23	Optional mood tag: 😊 Worth it / 😐 Neutral / 😞 Regret	P0
FR-24	Edit / delete any entry (delete is a soft delete with undo snackbar; purged after 30 days)	P0
FR-25	Recent notes as one-tap suggestions (e.g. "Groceries")	P1
FR-26	Log extra income (bonus, gift) mid-month	P1
FR-27	Target: save an entry in ≤ 3 taps and ≤ 5 seconds	P0
FR-28	**Refund** entry type: positive amount stored with `type = refund`, counted as negative spend in its pillar; optional link to the original expense	P0
FR-29	**Split** one purchase across pillars (e.g. supermarket bill = Needs + Wants) as linked entries sharing a `split_group_id`	P2
FR-84	Undo for edits as well as deletes (snackbar restores previous values)	P1

6.4 Dashboard

ID	Requirement	Priority
FR-30	Show remaining spendable for the month and per-day allowance	P0
FR-31	Savings-goal progress (projected savings if current pace continues)	P0
FR-32	4 pillar bars: spent vs planned, colour shifts as limit approaches	P0
FR-33	Today's entries and last 5 transactions	P0
FR-34	Days left in period	P0
FR-35	**Overspend state:** when remaining < 0, show "Over by Rs X — this reduces your savings to Rs Y"; daily allowance shows Rs 0 (never negative); tone is supportive, never shaming	P0
FR-36	**My stats** screen computed from the local DB: logging streak, % days logged, avg entries/day, goals met, savings rate trend, reflections completed	P0
FR-37	Dashboard shows the current **improvement** chosen last month and a "Plan not set" card if the period is in `planning` status (logging is still allowed)	P0

6.5 Weekly Check-in

ID	Requirement	Priority
FR-40	Weekly notification opens a summary of the week by pillar	P0
FR-41	Short prompt: "Are you on track? One thing to watch this week?" with free-text note	P0
FR-42	Weeks start on a user-chosen weekday (default Monday) and are **clipped to the period** (first/last week of a period may be shorter)	P0

6.6 Monthly Reflection (Question 4)

ID	Requirement	Priority
FR-50	At period end, guided reflection wizard opens	P0
FR-51	Shows: income, fixed, total variable by pillar, actual savings vs goal	P0
FR-52	Answers 4 reflection prompts (free text)	P0
FR-53	Shows "Regret" list and "Worth it" list from mood tags	P1
FR-54	User sets ONE improvement for next month; it appears on next month's dashboard	P0
FR-55	Reflection is saved and viewable in History	P0
FR-56	Next month's setup is pre-filled from this month	P0
FR-57	Reflection asks "Did I keep last month's improvement?" (Yes / Partly / No) before choosing a new one	P0
FR-58	**Editing a closed period:** totals are recomputed until the reflection is completed; after that the reflection snapshot is frozen and the month shows "Edited after close: ±Rs X"	P0
FR-59	If the goal was **missed**, the reflection shows by how much, which pillar overshot most, and suggests (not forces) a smaller goal or a pillar cap	P1

6.7 History & Insights

ID	Requirement	Priority
FR-60	List of past months with savings achieved and goal hit/miss	P0
FR-61	Month-over-month bar chart of spend per pillar	P1
FR-62	Savings trend line over 12 months	P1
FR-63	Total lifetime savings (Σ actual savings of closed periods, per §2.3)	P1
FR-64	Yearly review: 12-month totals per pillar, best/worst month, yearly reflection prompt	P2

6.8 Data Management

ID	Requirement	Priority
FR-70	Export transactions to CSV (share sheet)	P0

ID	Requirement	Priority
FR-71	Full backup to a JSON file; restore from file	P0
FR-72	Delete all data (with double confirmation)	P0
FR-73	**Automatic weekly backup** to a user-chosen folder via the Storage Access Framework (e.g. Downloads/Kakeibo); keep the last 8 files; dashboard warns if last backup is > 14 days old	P0
FR-74	**Restore** validates file format and `schemaVersion`, takes a safety snapshot of current data first, then fully replaces data (no merge); rejects files from a newer schema with a clear message	P0
FR-75	CSV export escapes cells starting with `=`, `+`, `-`, `@` (formula injection); sharing a backup shows a "this file is not encrypted" warning	P0

6.9 Reminders

ID	Requirement	Priority
FR-80	Daily reminder at a user-chosen time. Implementation: reminders are pre-scheduled for the next 7 days; saving an entry cancels **today's** reminder; every app open re-schedules the 7-day window	P0
FR-81	Weekly check-in reminder	P0
FR-82	Month-start "plan your month" and month-end "reflect" reminders	P0
FR-83	If notifications are denied or disabled, the same prompts appear as in-app cards (plan, check-in, reflect, backup)	P0
FR-85	Optional evening "Anything else today? (cash?)" reminder	P1
FR-86	Ask for notification permission (Android 13+) in onboarding with a one-line explanation first; suggest excluding the app from battery optimisation if reminders are missed	P0

6.10 Onboarding Guide & In-App Help

New users must learn **both** the Kakeibo method **and** how to use the app. The guide is layered: a short first-run tour, learn-by-doing on real screens, and a Help Center that is always available.

6.10.1 First-run guide (welcome tour)

ID	Requirement	Priority
FR-90	Welcome tour of 5 swipeable cards with illustrations: (1) What is Kakeibo, (2) The 4 questions, (3) The 4 pillars with examples, (4) The monthly cycle Plan → Log → Reflect, (5) Your data stays on your phone	P0
FR-91	Each card has Next / Back and a progress dots indicator; **Skip** is always visible; skipping still leads to the required setup (currency, start day)	P0
FR-92	Tour content stored locally (no network) and is localisable	P0
FR-93	Guide can be replayed anytime from Settings → Help → "Replay guide"	P0

6.10.2 Guided first actions (learn by doing)

ID	Requirement	Priority
FR-94	After setup, a **guided "Plan your first month"** walkthrough explains each field inline (what income/fixed/savings mean, with an example)	P0
FR-95	**Guided first expense:** coach mark points to the + button, then to the amount field, then to the pillar selector with a hint "Which pillar? Ask: do I *need* it?"	P0
FR-96	A **pillar quiz** (optional, 4–6 sample items like "Netflix", "Bus ticket", "Birthday gift") teaches categorising; shows the suggested pillar and why	P1
FR-97	A **"Getting started" checklist** card on the dashboard: Plan month ✓ · Log first expense · Set reminder · Do first weekly check-in · Make a backup. Dismissible once complete	P0
FR-98	Option to explore with **sample/demo data** (clearly labelled, removable in one tap, never mixed with real data)	P2

6.10.3 Contextual coach marks & tips

ID	Requirement	Priority
FR-99	First visit to each major screen (Dashboard, Transactions, Plan, Reflection, History) shows 1–3 coach marks highlighting key controls; each shown **once** and tracked in settings	P0
FR-100	Small **(?) info icons** next to key numbers (Spendable, Daily allowance, Projected savings) open a bottom sheet explaining the formula in plain words	P0
FR-101	Gentle **"Kakeibo tip of the day"** on the dashboard (rotating, offline list of ~30 tips), can be turned off	P1
FR-102	First reflection ever is **guided**: explains why each question matters before asking it	P0
FR-103	Meaningful **empty states** with guidance on every list/chart (e.g. "No expenses yet — tap + to log your first one")	P0

6.10.4 Help Center

ID	Requirement	Priority
FR-104	Settings → **Help Center** with sections: Kakeibo basics, Using the app (one article per feature), FAQ, Backup & restore how-to, Privacy	P0
FR-105	Offline, searchable help articles stored as bundled Markdown assets	P1
FR-106	FAQ covers at least: "I started mid-month", "I overspent a pillar", "I missed my savings goal", "How do I log a refund?", "I forgot my PIN", "How do I move to a new phone?"	P0
FR-107	Accessibility: tour and coach marks work with TalkBack (announced text, focus order) and respect large fonts; animations respect "remove animations" setting	P0
FR-108	Guide & Help content must explain: starting mid-month (partial period), fixed vs variable costs, which pillar to use for ambiguous items, why backups matter and how auto-backup works, the PIN recovery code, why notification / battery permissions are requested, and that a missed reflection never blocks logging	P0

Acceptance criteria (guide): - A first-time user can complete tour + setup + first expense in **under 3 minutes**. - After the tour, the user can explain the 4 pillars (validated in personal testing). - Every coach mark

is dismissible by tap anywhere, never blocks input for more than one step, and never re-appears after dismissal. - Replay guide resets tour and coach-mark flags without touching user data.

Technical notes: coach marks via an overlay package (e.g. `showcaseview` or `tutorial_coach_mark`); help articles in `assets/help/*.md` rendered with `flutter_markdown`; per-guide flags stored in the `settings` table (`guide_tour_done`, `coach_<screen>_seen`, `checklist_<item>_done`).

---

7. User Flows

7.1 First launch

```
Splash -> Welcome guide (5 cards, skippable) -> Currency & month start
       -> Notification permission (with reason) -> Set PIN + recovery code (optional)
       -> Partial or full first month? -> Guided "Plan your first month"
       -> Dashboard + coach marks -> Guided first expense
       -> "Getting started" checklist stays on dashboard until complete
```

7.2 Logging an expense (the 5-second flow)

```
Dashboard -> [+] FAB -> type amount -> tap pillar (Needs/Wants/Culture/Unexpected)
          -> (optional note / mood) -> Save -> Dashboard updates instantly
```

7.3 Month rollover

```
Period end date reached
   └-> Notification "Time to reflect"
         └-> Reflection wizard
               ├- Summary numbers
               ├- 4 questions
               ├- Regret / Worth-it review
               └- Pick 1 improvement
                     └-> Plan next month (pre-filled) -> New period starts
```

If the user ignores the reflection, a new period still starts automatically (so logging is never blocked), and the pending reflection remains available as a card on the dashboard.

---

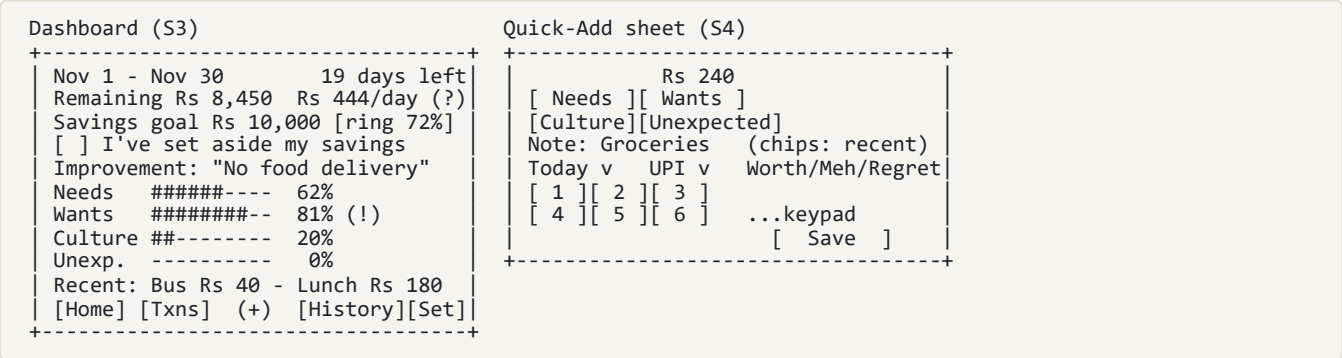
8. Screen Inventory

#	Screen	Key elements
S1	Intro / Onboarding	Kakeibo explanation, currency, start day
S2	Lock screen	PIN pad, biometric prompt
S3	Dashboard (Home)	Remaining spendable, savings ring, 4 pillar bars, recent list, FAB
S4	Quick-Add sheet	Amount keypad, 4 pillar buttons, note, date, mood
S5	Transactions	Grouped by day, filter by pillar/mood, search

#	Screen	Key elements
S6	Month Plan	Income list, fixed list, savings goal, pillar split, live spendable
S7	Weekly Check-in	Week totals, note field
S8	Reflection Wizard	Summary → questions → mood review → improvement
S9	History	Month cards, charts
S10	Month Detail	Past plan, totals, reflection answers
S11	Settings	Currency format (symbol only), pillars, reminders, theme, lock, backup/export, about Kakeibo
S12	Welcome Guide	5 illustrated cards, progress dots, Skip / Next
S13	Help Center	Search, Kakeibo basics, feature guides, FAQ, Replay guide
S14	Help Article	Rendered Markdown article, related articles
S15	Pillar Quiz	Sample items → pick pillar → explanation

Navigation: bottom bar with **Home · Transactions · History · Settings** + central FAB for quick-add.

8.1 Wireframe notes



Pillar state is never shown by colour alone: bars also show % text and an icon (! / ✕) at caution and over states.

8.2 Empty & error states

Situation	What the user sees
No plan for current period	Card "Plan your month" + button; logging still works
No expenses yet	Illustration + "Tap + to log your first expense"
Income = 0	Plan shows warning; spendable and allowance hidden, totals still tracked
Pillar budget = 0 but spending exists	Bar shows spent amount with "no budget set" label
Budgets don't sum to Spendable	Inline note "Rs X unassigned" / "Rs X over-assigned"
No history yet	"Your first month review will appear here"
Notifications denied	In-app cards replace reminders (FR-83) + link to settings

Situation	What the user sees
Backup older than 14 days / never	Dashboard banner "Back up your data"
Restore failed / newer schema	Error dialog with reason; current data untouched (snapshot kept)
DB error on write	Snackbar "Couldn't save — try again"; error logged locally (NFR-12)

Design language: calm, paper-like, Japanese-minimal — off-white background, ink-dark text, four soft accent colours (one per pillar), generous whitespace, rounded cards, serif headings optional to evoke the notebook feel.

---

9. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Offline	100% of features work without internet; no INTERNET permission in v1
NFR-02	Performance	Cold start < 1.5 s on mid-range device; save entry < 100 ms
NFR-03	Size	Release APK < 25 MB (split per ABI)
NFR-04	Compatibility	Android 8.0 (API 26) and above
NFR-05	Reliability	No data loss on crash; all writes transactional
NFR-06	Privacy	No analytics, no ads, no third-party trackers
NFR-07	Accessibility	TalkBack labels, ≥ 4.5:1 contrast, scalable fonts (layout works at 200%), touch targets ≥ 48 dp, state never conveyed by colour alone
NFR-08	Localization-ready	All strings externalised; number formatting by locale; currency symbol taken from the user's format setting (FR-109), never hard-coded
NFR-09	Money accuracy	Amounts stored as integers in minor units (cents) — never floating-point
NFR-10	Maintainability	≥ 80% unit-test coverage on domain logic
NFR-11	Release safety	Signing keystore + passwords backed up in 2 offline places (losing it means updates require uninstall = data loss); release checklist includes "back up data before any reinstall"
NFR-12	Diagnostics	Global error handler (`FlutterError.onError`, `PlatformDispatcher.onError`) writes to a rotating local log file (max 1 MB × 3); exportable from Settings → Help → "Export diagnostics"
NFR-13	Currency	LKR uses 2 decimal places (cents); amounts stored as integer cents. Grouping is western style (`Rs 1,250,000.00`, locale `en_LK` / `si_LK`). Money is formatted by one `MoneyFormatter` that reads `currency_symbol`, `currency_symbol_position` and `currency_symbol_space`; changing the *symbol format* is always allowed (display only), changing the *currency itself* is out of scope for v1
NFR-14	Time	All dates are local calendar dates (`yyyy-MM-dd`); timestamps stored UTC; device timezone from `flutter_timezone`; a date moved backwards never deletes or re-closes periods

10. Technical Architecture

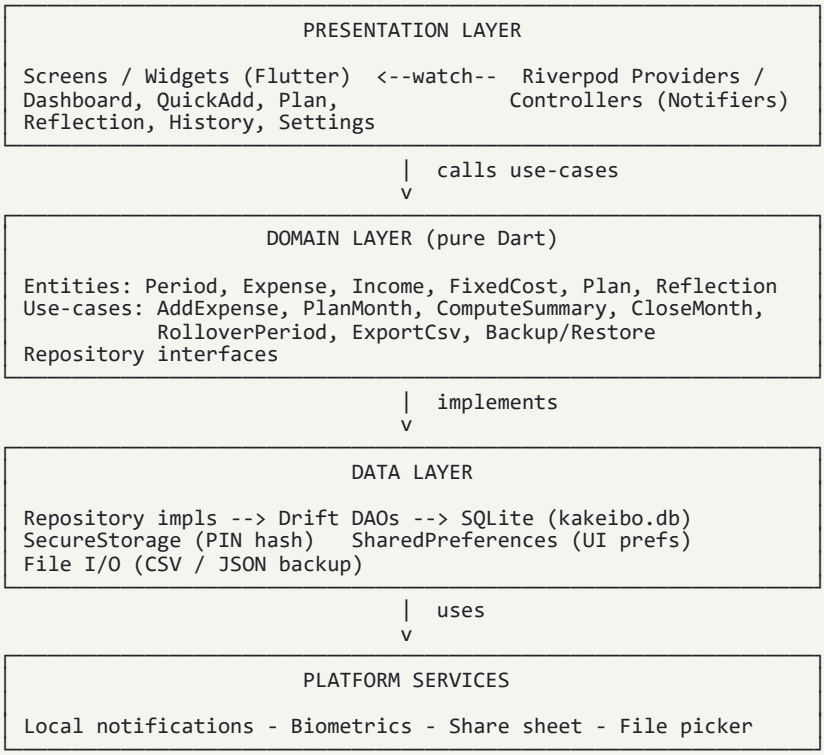
10.1 Technology choice

Concern	Choice	Why
Framework	**Flutter 3.x (Dart)**	One codebase, fast UI, produces an Android APK directly, iOS possible later, excellent chart libraries
Architecture	**Clean Architecture + feature-first folders**	Testable domain logic, isolated data layer
State management	**Riverpod**	Compile-safe, testable, no BuildContext coupling
Local database	**Drift (SQLite)**	Typed SQL, reactive streams, migrations, transactions
Secure storage	**flutter_secure_storage**	PIN hash stored in Android Keystore-backed storage
Biometrics	**local_auth**	Fingerprint / face unlock (requires `MainActivity` to extend `FlutterFragmentActivity` and `USE_BIOMETRIC` permission)
Notifications	**flutter_local_notifications** + `timezone` + `flutter_timezone`	Offline reminders via `zonedSchedule` with `AndroidScheduleMode.inexactAllowWhileIdle` (no `SCHEDULE_EXACT_ALARM` needed); `RECEIVE_BOOT_COMPLETED` to restore after reboot; `POST_NOTIFICATIONS` on Android 13+
Background work	**None in v1**	Rollover runs on app start/resume; reminders are pre-scheduled, so WorkManager is not required
Onboarding / help	**showcaseview** (or tutorial_coach_mark), **flutter_markdown**	Coach marks, bundled help articles
Storage access	**Storage Access Framework** (via `file_picker` / `shared_storage`)	Auto-backup folder chosen once by user
Logging	**logging** + rotating file sink	Local diagnostics (NFR-12)
Charts	**fl_chart**	Lightweight, customisable
Navigation	**go_router**	Declarative routes, deep links from notifications
Serialization	**freezed + json_serializable**	Immutable models, backup JSON
File export	**share_plus**, **file_picker**	CSV/JSON export & restore
i18n	**intl** + ARB files	Currency/date formatting

> **Alternative considered:** Native Kotlin + Jetpack Compose + Room. Better if the app will be Android-only forever. Flutter chosen for speed of development and future iOS option. React Native/Expo is also

> viable but SQLite + charts + notifications are smoother in Flutter.

10.2 High-level architecture



10.3 Project structure

```
kakeibo/
├── lib/
│   ├── main.dart
│   ├── app/                    # MaterialApp, router, theme
│   │   ├── router.dart
│   │   └── theme.dart
│   ├── core/
│   │   ├── money/              # Money value object (int cents) + MoneyFormatter (symbol format, FR-109)
│   │   ├── date/               # Period calculation helpers
│   │   ├── db/                 # Drift database, tables, migrations
│   │   └── services/           # notifications, auth, file io
│   └── features/
│       ├── onboarding/         # welcome tour, setup, guided first actions
│       ├── guide/              # coach marks, tips, checklist, help center
│       ├── plan/               # monthly setup (Q1, Q2)
│       ├── expenses/           # quick add, list (Q3)
│       ├── dashboard/
│       ├── checkin/            # weekly
│       ├── reflection/         # month-end (Q4)
│       ├── history/
│       └── settings/
│           (each feature: data/ domain/ presentation/)
├── test/                       # unit + widget tests
├── integration_test/
├── android/                    # Gradle, signing config
└── pubspec.yaml
```

10.4 Data flow example — adding an expense

```
QuickAddSheet ──(amount, pillar, note)──> ExpenseController.add()
      ──> AddExpenseUseCase  (validate > 0, resolve current Period)
      ──> ExpenseRepository.insert()  ──> Drift INSERT (transaction)
      ──> Drift stream emits ──> dashboardSummaryProvider recomputes
      ──> Dashboard rebuilds (remaining, pillar bars, savings projection)
      ──> NotificationService.cancelTodayReminder()
```

10.5 Reactive state

- `currentPeriodProvider` — the active Period **read from the stored `periods` row** whose dates contain today (created by rollover); it is never recomputed from today + start day, so changing settings cannot shift existing periods.
- `periodSummaryProvider(periodId)` — stream of computed totals from Drift queries.
- `expensesProvider(filter)` — stream of filtered expenses.
- Controllers (`AsyncNotifier`) handle writes; reads are streams, so the UI is always consistent with the DB.

---

11. Data Model

All money fields are **INTEGER in minor units** (Rs 125.50 → `12550`).

11.1 Entity-relationship overview

```
Settings (1)

Period (1) ──< Income (*)
       (1) ──< FixedCost (*)
       (1) ──< PillarBudget (4)
       (1) ──< Expense (*) >── Pillar (4)
       (1) ──  Reflection (0..1)
       (1) ──< WeeklyCheckin (*)
```

11.2 Tables

pillars

Column	Type	Notes
id	INTEGER PK	1=Needs, 2=Wants, 3=Culture, 4=Unexpected
name	TEXT	user-renamable
color	INTEGER	ARGB
icon	TEXT	icon key
sort_order	INTEGER	

periods

Column	Type	Notes
id	INTEGER PK	
start_date	TEXT (ISO date)	inclusive
end_date	TEXT (ISO date)	inclusive
savings_goal	INTEGER	minor units (resolved amount)
savings_goal_type	TEXT	`amount` / `percent`
savings_goal_pct	INTEGER	basis points (e.g. 2000 = 20%), nullable
savings_set_aside	BOOLEAN	FR-18 "save first" ticked
is_partial	BOOLEAN	FR-17 first partial period
status	TEXT	`planning` / `active` / `closed`
created_at, updated_at	TEXT	

> The improvement for a period is read from the **previous period's reflection** (`reflections.improvement`) — stored once, not duplicated.

incomes

Column	Type	Notes
id	INTEGER PK	
period_id	FK → periods	
label	TEXT	"Salary", "Bonus"
amount	INTEGER	
date	TEXT	
is_planned	BOOLEAN	true = entered in plan; false = extra income logged mid-month (FR-26)

fixed_costs

Column	Type	Notes
id	INTEGER PK	
period_id	FK → periods	
label	TEXT	"Rent", "Phone"
amount	INTEGER	
is_recurring	BOOLEAN	copy to next period

pillar_budgets

Column	Type	Notes
period_id	FK	composite PK
pillar_id	FK	composite PK
planned_amount	INTEGER	

expenses

Column	Type	Notes
id	INTEGER PK	
period_id	FK → periods	indexed
pillar_id	FK → pillars	indexed
type	TEXT	`expense` / `refund` (FR-28)
amount	INTEGER	> 0 (sign derived from `type`)
note	TEXT	nullable
date	TEXT	local date
mood	TEXT	`worth` / `neutral` / `regret` / null
payment_method_id	FK → payment_methods	nullable
refund_of_id	FK → expenses	nullable
split_group_id	TEXT	nullable (FR-29)
is_lump_sum	BOOLEAN	"Before I started" entry (FR-17)
deleted_at	TEXT	soft delete (FR-24), nullable
created_at, updated_at	TEXT	

Indexes: `(period_id, date)`, `(period_id, pillar_id)`, `(deleted_at)`.

payment_methods — id, name, icon, is_hidden, sort_order.

plan_changes (FR-16)

Column	Type	Notes
id	INTEGER PK	
period_id	FK	
field	TEXT	e.g. `savings_goal`, `pillar_budget:2`
old_value, new_value	TEXT	
changed_at	TEXT	UTC

reflections

Column	Type	Notes
period_id	PK, FK	one per period
saved_amount	INTEGER	snapshot
goal_met	BOOLEAN	snapshot
answer_q1..q4	TEXT	prompts answers
prev_improvement_kept	TEXT	`yes` / `partly` / `no` (FR-57)
improvement	TEXT	ONE change for next month
completed_at	TEXT	null until completed; after it is set, the snapshot is frozen (FR-58)
edited_after_close	INTEGER	net amount change after freeze, default 0

weekly_checkins

Column	Type	Notes
id	INTEGER PK	
period_id	FK	
week_start	TEXT	
note	TEXT	

settings (key-value) — defined keys:

Key	Example
`currency_code`	`LKR` (fixed in v1)
`currency_symbol` / `currency_symbol_position` / `currency_symbol_space`	`Rs` / `before` / `true` (FR-109)
`month_start_day` / `pending_month_start_day`	`1` / `25` (applies from next period)
`week_start_day`	`1` (Monday)
`theme`	`system`
`reminder_daily_time`, `reminder_evening_time`, `reminder_weekly_day_time`	`21:00`, `22:30`, `SUN 19:00`
`reminders_enabled.<type>`	`true`
`lock_enabled`, `lock_timeout_sec`, `biometric_enabled`, `flag_secure`	`true`, `60`, `true`, `false`
`backup_folder_uri`, `last_backup_at`, `auto_backup_enabled`	SAF URI, ISO timestamp, `true`
`tips_enabled`, `guide_tour_done`, `coach_<screen>_seen`, `checklist_<item>_done`	booleans

PIN hash, salt, failed-attempt counter and recovery-code hash live in **secure storage**, not in `settings`.

11.3 Database rules & migrations

- `PRAGMA foreign_keys = ON` at open. Deleting a period cascades to its incomes, fixed costs, budgets, expenses, check-ins and reflection; pillars and payment methods are **restricted** (hide instead of delete).
- Schema version is owned by Drift (`schemaVersion`); no separate settings key.
- Every schema change: bump version, add a step in `MigrationStrategy.onUpgrade`, commit a `drift_dev schema dump` JSON, and add a generated migration test for each step (v1→v2, v2→v3 …).
- Soft-deleted rows older than 30 days purged on app start.

11.4 Backup JSON format

```
{
  "app": "kakeibo",
  "schemaVersion": 1,
  "exportedAt": "2026-11-03T10:00:00Z",
  "settings": { "...": "..." },
  "pillars": [ ],
  "periods": [
    { "id": 1, "startDate": "2026-11-01", "endDate": "2026-11-30",
      "incomes": [], "fixedCosts": [], "pillarBudgets": [],
      "expenses": [], "reflection": null, "checkins": [] }
  ]
}
```

---

12. Core Business Logic

12.1 Period calculation

Given `monthStartDay = d` (1–28), the **first** period is computed as:

```
if today.day >= d:  start = (today.year, today.month, d)
else:               start = (previous month, d)
end = start + 1 month − 1 day
```

- Days 29–31 are excluded from the start-day choice to avoid ambiguous months.
- Partial first period (FR-17a): `start = install date`, `end` as above; pro-rate factor = `daysInPartial / daysInFullPeriod`, applied to suggested income, fixed and savings (user can edit).
- After creation, periods are **stored rows**; later periods always start the day after the previous `end_date`.
- **Changing month start day** stores `pending_month_start_day`; it applies when the *next* period is created — the current period is either shortened or extended (max 45 days) so the next one starts on the new day. Existing periods are never re-dated.

12.2 Summary calculations

All values in integer minor units. `expenses` excludes soft-deleted rows.

```
income         = Σ incomes.amount                      (planned + extra)
fixed          = Σ fixed_costs.amount
savingsGoal    = amount  OR  floor(income × pct / 10000)
spendable      = income − fixed − savingsGoal
spent          = Σ expense.amount − Σ refund.amount
remaining      = spendable − spent
daysLeft       = max(1, end_date − today + 1)
dailyAllowance = floor(max(0, remaining) / daysLeft)
actualSavings  = income − fixed − spent                (may be negative)
daysElapsed    = max(1, today − start_date + 1)
projectedSpend = spent × totalDays / daysElapsed       (show only after day 3)
projectedSave  = income − fixed − projectedSpend
pillarUsage%   = planned_pillar > 0 ? spent_pillar / planned_pillar : n/a
unassigned     = spendable − Σ planned_pillar          (warn if ≠ 0)
overspend      = max(0, −remaining)                    (FR-35)
```

12.3 Pillar bar status

Usage	State
< 75%	Calm (pillar colour) + % text
75 – 100%	Caution (amber) + "!" icon
> 100%	Over (soft red) + "✕" icon + gentle message, never shaming
no budget	Neutral bar with spent amount and "no budget set"

12.4 Rollover

Runs on **app start and every resume** (no background job). It is **idempotent** — running it twice changes nothing.

1. Load the latest stored period. While `today > period.end_date`: - mark it `closed` (reflection may still be pending), - create the next period starting `end_date + 1` (applying any `pending_month_start_day`), - copy recurring fixed costs, savings goal (amount or %) and pillar split; status = `planning` until the user confirms the plan.
2. If several periods were missed (app unused for months), each one is created so history stays continuous; empty ones are marked "no activity".
3. Show the pending reflection card and (re)schedule notifications for the next 7 days.
4. All steps run in **one DB transaction**.

Logging is allowed in `planning` periods; the dashboard shows a "Plan not set" card (FR-37).

12.5 Validation & editing rules

- Amount > 0 and ≤ 10,000,000 major units.
- Savings goal ≤ income − fixed (warn otherwise, allow override).
- Expense date may be any date inside a **stored** period, including the current partial period. Dates before the first period are rejected with a hint to use the "Before I started" lump sum.
- Editing entries of a closed period: totals recompute until its reflection is completed; afterwards the snapshot is frozen and `edited_after_close` is updated (FR-58).
- Refunds cannot exceed the linked original expense amount (if linked).

---

13. Security & Privacy

- **No internet permission** in v1 → data cannot leave the device except via user-initiated export/share or the user-chosen backup folder.
- **App lock:** PIN (4–6 digits) hashed with PBKDF2-HMAC-SHA256 (≥ 100k iterations) + random salt, stored in `flutter_secure_storage` (Android Keystore). Biometric unlock via `local_auth`.
- **PIN recovery (FR-07):** a one-time recovery code shown at setup (hash stored in secure storage). Forgotten PIN → biometric or recovery code → set new PIN. No code and no biometric → *Reset app* wipes data after warning and offers to restore from a backup.
- **Brute force (FR-08):** escalating lockout after 5 wrong attempts; counter persisted in secure storage so restarting the app does not reset it.
- **Auto-lock & re-lock on resume** after the configured timeout (FR-09); optional `FLAG_SECURE` hides content in the app switcher and blocks screenshots.
- **Database:** in v1 data resides in app-private storage; SQLCipher encryption is a P2 option.
- **Backups:** plain JSON in v1 with a warning before sharing (FR-75); password-protected backups (AES-GCM, key from PBKDF2) in v1.1.
- **Android system backup — fixed decision:** `android:allowBackup="false"` plus `dataExtractionRules` excluding everything. Reason: the manifest cannot be changed at runtime, and silently restoring a DB without the matching Keystore-protected PIN would break the lock. Our own backup files (FR-71/73) are the supported way to move to a new phone.
- **CSV injection:** cells beginning with `= + - @` are prefixed with `'` on export.

---

14. Testing Strategy

Level	Tooling	Focus
Unit	`flutter_test`, `mocktail`	Money math, period calculation, rollover, summaries, refunds, CSV/JSON mapping
Database	Drift in-memory DB	DAOs, constraints, cascades, **migration tests per schema step**
Widget	`flutter_test`	Quick-add, dashboard states (incl. empty/overspend), reflection wizard, coach marks
Integration	`integration_test` on emulator	Onboarding → plan → add → rollover → reflect → backup → restore
Manual	Real device checklist	Notifications after reboot, permission denied, biometrics, PIN recovery, export/share, dark mode, TalkBack, 200% font

Key edge cases: month-start day 28 in February, leap years, timezone change, device date moved backwards, app unused for 3 months, partial first period, empty month, income = 0, refund larger than pillar spend, restore of older and newer schema version, start-day change mid-period.

14.1 Acceptance criteria (key P0 requirements)

FR	Given / When / Then
FR-20/27	**Given** the dashboard, **when** I tap +, type 240, tap Needs and Save, **then** the entry is stored, the Needs bar and Remaining update within 100 ms, and it took ≤ 3 taps besides typing
FR-14	**Given** income 50,000, fixed 20,000, savings 10,000, **when** I view the plan, **then** Spendable shows 20,000 and updates live as I edit any field
FR-17	**Given** first install on the 18th with start day 1, **when** I pick "partial period", **then** the period is 18th – end of month and suggested amounts are pro-rated
FR-35	**Given** remaining = −1,500, **then** the dashboard shows "Over by Rs 1,500", daily allowance Rs 0 and the reduced savings figure
FR-50/54	**Given** a period ended, **when** I open the app, **then** a new period exists, the reflection card is shown, and completing it puts my improvement on the new dashboard
FR-109	**Given** expenses exist, **when** I change the symbol from `Rs` to `LKR` and position to *after*, **then** every amount shows like `12,500.00 LKR` immediately and all stored amounts and totals are unchanged
FR-07	**Given** I forgot my PIN, **when** I enter the recovery code, **then** I can set a new PIN and all data is intact
FR-73/74	**Given** auto-backup is on, **when** 7 days pass and I open the app, **then** a new backup file exists in the chosen folder; **when** I restore it on a fresh install, **then** all periods and entries match
FR-80	**Given** a 21:00 reminder, **when** I log an expense at 13:00, **then** no reminder fires today; **when** I log nothing, it fires at ~21:00, including after a reboot
FR-90/93	**Given** first launch, **when** I skip the tour, **then** I still reach setup; **when** I tap "Replay guide", **then** the tour and coach marks show again and no data changes

14.2 Definition of Done (per feature)

- All P0 FRs of the feature implemented and their acceptance criteria pass.
- Unit tests for domain logic (≥ 80% coverage), widget test for the main screen state(s).
- Empty, error and overspend states implemented (§8.2).
- Strings externalised; works in light/dark, TalkBack and 200% font.
- No new analyzer warnings; `flutter test` green; migration test added if the schema changed.
- Help article / coach marks updated if the feature is user-facing (FR-104, FR-108).
- Manually verified on a real Android device from a release build.

---

15. Build, Release & APK Distribution

15.1 Prerequisites

- Flutter SDK (stable), Android Studio + Android SDK (API 34/35), JDK 17
- A signing keystore (`keytool -genkey -v -keystore kakeibo-release.jks ...`) kept **outside** the repo; referenced via `android/key.properties` (git-ignored)
- **Back up the keystore and its passwords in two offline places** (NFR-11). A sideloaded app can only be updated with the same key; a lost key forces uninstall, which deletes app data.

15.2 Commands

```
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter test
flutter build apk --release --split-per-abi
# output: build/app/outputs/flutter-apk/app-arm64-v8a-release.apk
```

15.3 Configuration

- `applicationId`: `com.<yourname>.kakeibo`
- `minSdk 26`, `targetSdk` latest stable
- R8/ProGuard shrinking enabled, obfuscation with `--obfuscate --split-debug-info`
- Version: `versionName 1.0.0`, `versionCode` incremented per build

15.4 Distribution

- Personal use: sideload the arm64 APK (enable "Install unknown apps").
- **Release checklist:** run tests → bump `versionCode` → build signed APK → make an in-app backup on the phone → install over the existing app (never uninstall) → verify data and reminders.
- Later: Google Play (AAB via `flutter build appbundle`) or F-Droid.

---

16. Roadmap & Milestones

Milestone	Duration	Deliverables
M0 — Setup	2–3 days	Flutter project, folder structure, theme, router, Drift DB, CI lint/test
M1 — Core ledger	1 week	Period logic, plan-month wizard, quick-add, transactions list
M2 — Dashboard	4–5 days	Summary calculations, pillar bars, savings projection
M3 — Ritual	1 week	Rollover, reflection wizard, weekly check-in, improvement goal
M4 — Insights & data	4–5 days	History, charts, CSV export, JSON backup/restore
M5 — Polish & security	4–5 days	Reminders + permission flow, app lock + PIN recovery, auto-backup, My stats, diagnostics log, dark mode, accessibility
M5b — Onboarding guide	4–5 days	Welcome tour, guided first month & expense, coach marks, checklist, Help Center + FAQ
M6 — Release	2–3 days	Testing pass, signed release APK, install on device

Total MVP estimate: ~6–7 weeks for one developer working part-time-to-full-time.

---

17. Success Metrics (personal & product)

All metrics are computed **on-device** from the database and shown in *My stats* (FR-36); there is no analytics service.

Metric	Target after 3 months	How measured
Months with savings goal met	≥ 2 of 3	`reflections.goal_met`
Logging consistency	≥ 80% of days have an entry	distinct entry dates / days
Reflection completion	100% of closed months	reflections with `completed_at`
Avg time to log an expense	≤ 5 seconds	sheet-open → save duration, kept locally (last 100)
Savings rate trend	Rising month-over-month	actualSavings / income per period
Crash-free use	No unhandled errors in a month	local error log (NFR-12)

18. Risks & Mitigations

Risk	Impact	Mitigation
Logging fatigue → user abandons	High	5-second quick-add, smart note suggestions, home widget (v1.1), gentle reminders, streaks
Forgetting cash expenses	Medium	Evening "anything else today?" reminder
Data loss (phone lost/reset)	High	Automatic weekly backup to user folder (FR-73), stale-backup banner, safe restore (FR-74); encrypted Drive backup later
Forgotten PIN locks user out	High	Recovery code + biometric (FR-07); reset only as last resort
Lost signing key	High	Keystore backed up offline twice (NFR-11)
Floating-point money bugs	High	Integer minor units + Money value object + tests
Period edge cases (Feb, timezones)	Medium	Restrict start day to 1–28; exhaustive unit tests
Scope creep (bank sync etc.)	Medium	Non-goals list; stay true to Kakeibo philosophy
Notification restrictions (Android 13+/battery optimizers)	Medium	POST_NOTIFICATIONS flow (FR-86), inexact scheduling, reboot receiver, in-app fallback cards (FR-83)

19. Open Questions

1. ~~Default currency~~ **Resolved:** LKR, displayed `Rs`, symbol format changeable in Settings (FR-109).
2. Should the 4 default pillar names be in English only, or also show the Japanese terms?
3. Should the app support multiple people (e.g. family) in v1? (Assumed: no — §2.3.)
4. Should UPI/SMS-based suggested entries ever be added, given the mindful-entry philosophy?
5. Preferred PIN length: 4 or 6 digits?

Resolved: savings carry-over → §2.3 (per period + lifetime total; separate savings "jars" in v1.x). Home-screen widget → v1.1 (§5.2). Visual style → Japanese-minimal paper look (§8).

---

20. Sources

- SoFi — *Guide to Kakeibo: The Japanese Budgeting Method* — https://www.sofi.com/learn/content/kakeibo/
- Big Think — *Kakeibo money* — https://bigthink.com/smart-skills/kakeibo-money/
- Credit.com — *Kakeibo: The Mindful Japanese Budgeting System* — https://credit.com/blog/kakeibo
- Pocket — *The art of kakeibo* — https://getpocket.com/explore/item/the-art-of-kakeibo-the-japanese-budgeting-system-that-could-change-your-financial-life
- PocketGuard — *Kakeibo Method Explained* — https://pocketguard.com/blog/kakeibo-method/
- MoneyLion — *Kakeibo Japanese Budgeting Method* — https://moneylion.com/learn/kakeibo-japanese-budgeting-method
- Kotak Neo — *Kakeibo: Japanese Budgeting Method* — https://www.kotakneo.com/investing-guide/articles/kakeibo-japanese-budgeting-method/
- Freenance — *Kakeibo for Mindful Spending* — https://freenance.io/budgeting/kakeibo-budgeting-method/
- The Penny Hoarder — *Try Kakeibo* — https://www.thepennyhoarder.com/?p=132488
- Money Crashers — *Kakeibo Japanese budgeting method* — https://www.moneycrashers.com/kakeibo-japanese-budgeting-method-save-money/
- Michael Ryan Money — *Mastering the Kakeibo Method* — https://michaelryanmoney.com/kakeibo-method-budgeting/
- Restless — *Kakeibo explained* — https://restless.co.uk/money/everyday-finance/kakeibo-the-japanese-budgeting-method-explained/
- Penguin Random House — *Kakeibo* by Fumiko Chiba — https://penguinrandomhouse.com/books/588474/kakeibo-by-fumiko-chiba
- Outlook Money — *Tired of budget apps? Try Kakeibo* — https://www.outlookmoney.com/personal-finance/tired-of-budget-apps-try-kakeibo-a-japanese-method-to-save-mindfully
- Hacker News Showcase — *Kakeibo: a simple budget tracking app* — https://hnshowcase.com/post/48280241/kakeibo-a-simple-budget-tracking-app-for-simple-people

Note: most available sources are blogs and finance-company articles rather than the original 1904 literature; category and question wording varies between them, so this document uses a synthesised, widely-agreed version of the method.